

Multi-Class Classification

Line-by-Line Teaching Guide | Obesity Risk Prediction Dataset

Approx. 15–20 Minute Classroom Session

What you will learn:

- What Multi-Class Classification is and when to use it
- How to scale features using `StandardScaler`
- How to convert text columns to numbers using One-Hot Encoding
- How to train Logistic Regression with One-vs-All (OvA) strategy
- How to train Logistic Regression with One-vs-One (OvO) strategy
- How to evaluate your model using `accuracy_score`
- Every line of code explained — syntax, meaning, and a real example

Section 1 — What is Multi-Class Classification?

In **binary classification** we predict one of two answers: Yes or No, 0 or 1, Churn or Stay. In **multi-class classification** the model must choose from **3 or more possible classes**.

This notebook's problem: Given a person's eating habits, age, height, weight, and lifestyle — predict their **obesity level**. There are exactly 7 possible answers (classes):

- Insufficient_Weight
- Normal_Weight
- Overweight_Level_I
- Overweight_Level_II
- Obesity_Type_I
- Obesity_Type_II
- Obesity_Type_III

Simple Analogy:

Binary classification is like a door that is either OPEN or CLOSED. Multi-class classification is like a traffic light — it can be RED, YELLOW, or GREEN. Here we have 7 colours instead of 2.

Section 2 — Importing Libraries

Every Python program starts by telling Python which external toolkits to load. These are called **libraries** or **packages**. Think of them like apps on your phone — you install them once, then import them when needed.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.multiclass import OneVsOneClassifier
from sklearn.metrics import accuracy_score

import warnings
warnings.filterwarnings('ignore')
```

Code Line	Plain-English Explanation
<code>import pandas as pd</code>	Loads the Pandas library — it lets us work with tables (DataFrames). The 'as pd' part gives it a short nickname so we type 'pd' instead of 'pandas' every time. Example: <code>pd.read_csv()</code> instead of <code>pandas.read_csv()</code>
<code>import numpy as np</code>	Loads NumPy — used for fast math on arrays (lists of numbers). Nickname 'np'. Example: <code>np.mean([1,2,3])</code> gives 2.0
<code>import matplotlib.pyplot as plt</code>	Loads the charting/plotting part of Matplotlib. Nickname 'plt'. Used to draw graphs. Example: <code>plt.show()</code> displays whatever chart you built.
<code>import seaborn as sns</code>	Loads Seaborn — a prettier chart library built on top of Matplotlib. Nickname 'sns'. Example: <code>sns.countplot()</code> draws a bar chart of category counts.
<code>from sklearn.model_selection import train_test_split</code>	Gets just one function — <code>train_test_split</code> — from sklearn's <code>model_selection</code> module. This function splits our data into training and testing portions. 'from X import Y' means: from library X, take only function Y. Saves memory vs importing the whole library.
<code>from sklearn.preprocessing import OneHotEncoder, StandardScaler</code>	Gets two data-cleaning tools from sklearn: <code>OneHotEncoder</code> (converts text to 0/1 numbers) and <code>StandardScaler</code> (normalises number ranges so they are

Code Line	Plain-English Explanation
	comparable).
<code>from sklearn.linear_model import LogisticRegression</code>	Gets the LogisticRegression model class — the main algorithm we will train.
<code>from sklearn.multiclass import OneVsOneClassifier</code>	Gets the OneVsOneClassifier wrapper — it lets us apply any binary classifier (like LogisticRegression) using the One-vs-One multi-class strategy.
<code>from sklearn.metrics import accuracy_score</code>	Gets the accuracy_score function — it compares the model's predictions to the correct answers and returns a number between 0 and 1 (e.g., 0.87 = 87% correct).
<code>import warnings warnings.filterwarnings('ig nore')</code>	Loads the warnings module and silences unimportant warning messages so the output screen stays clean and easy to read.

Syntax: import vs from...import

`import pandas as pd` → loads the ENTIRE pandas library and gives it the alias 'pd'

`from sklearn.metrics import accuracy_score` → loads ONLY the accuracy_score function from sklearn

Quick Rule: Use 'from X import Y' when you only need one or two things from a big library. It keeps your code lighter and cleaner.

Section 3 — Loading the Dataset

```
file_path = "https://cf-courses-data.s3.us.cloud-object-storage...csv"
data = pd.read_csv(file_path)
data.head()
```

Code Piece	Plain-English Explanation
<code>file_path = "https://...csv"</code>	Creates a variable called <code>file_path</code> and stores the URL of the CSV file inside it. A variable is just a named box that holds a value. Example: <code>file_path = 'data.csv'</code> → Now whenever you write <code>file_path</code>, Python knows you mean that CSV file.
<code>data = pd.read_csv(file_path)</code>	<code>pd.read_csv()</code> reads the CSV file from the given path or URL and creates a DataFrame — a table with rows and columns, exactly like an Excel sheet. The table is stored in the variable <code>'data'</code>. 2111 rows × 17 columns.
<code>data.head()</code>	<code>.head()</code> is a method (an action) that belongs to every DataFrame. It shows the first 5 rows of the table so you can quickly check whether the data loaded correctly. You can write <code>data.head(10)</code> to see the first 10 rows instead.

Section 4 — Exploratory Data Analysis (EDA)

Before touching any model, we always **look at** the data first. This is called EDA — it helps us understand what we are working with.

4.1 — Visualising the Target Variable

```
sns.countplot(y='NObeyesdad', data=data)
plt.title('Distribution of Obesity Levels')
plt.show()
```

Code Piece	What it does
<code>sns.countplot(y='NObeyesdad', data=data)</code>	Draws a horizontal bar chart. Each bar represents one obesity class and its length shows how many rows belong to that class. y='NObeyesdad' → the column to count; draws bars horizontally (y-axis). data=data → which DataFrame to use.
<code>plt.title('Distribution of Obesity Levels')</code>	Adds a text title above the chart. You can put any text inside the quotes.
<code>plt.show()</code>	Tells Python to actually render and display the chart. Without this line, the chart is built in memory but never shown on screen.

4.2 — Checking Data Quality (Exercise 1)

```
print(data.isnull().sum()) # counts missing values per column
print(data.info())        # column names, types, row count
print(data.describe())    # stats: mean, min, max, std for number
                           columns
```

Method	What it tells you
<code>data.isnull()</code>	Returns a True/False table. True = that cell is empty (null/missing). False = it has a value.
<code>.sum()</code>	Counts how many True values are in each column. If the answer is 0 for every column, there is NO missing data — great news! Example output: Gender 0, Age 0, Height 0 ... (all zeros means no missing values)

Method	What it tells you
<code>data.info()</code>	Prints a summary: column name, how many non-null values, and the data type (int64, float64, object). object means text/string. float64 means decimal number. int64 means whole number.
<code>data.describe()</code>	Prints statistical summary for all numerical columns: count, mean, std (standard deviation), min, 25th percentile, 50th (median), 75th percentile, max. Example: Age column → mean=24.3, min=14, max=61

Section 5 — Preprocessing the Data

Raw data cannot go directly into a machine learning model. We need to: (1) bring all numbers to the same scale, and (2) convert text into numbers. This section does both.

Step 1 — Feature Scaling (StandardScaler)

Why scale? Height values are 1.5–2.0. Weight values are 50–180. If we leave them raw, the model thinks weight is more important just because its numbers are bigger — which is wrong. Scaling makes all columns equally weighted.

```
continuous_columns =
data.select_dtypes(include=['float64']).columns.tolist()

scaler = StandardScaler()
scaled_features = scaler.fit_transform(data[continuous_columns])

scaled_df = pd.DataFrame(scaled_features,
                        columns=scaler.get_feature_names_out(continuous_columns))

scaled_data = pd.concat([data.drop(columns=continuous_columns),
scaled_df], axis=1)
```

Code Piece	Plain-English Explanation + Example
<code>data.select_dtypes(include=['float64'])</code>	Scans through all columns of the DataFrame and keeps only the ones whose data type is float64 — meaning decimal numbers like 1.62, 64.5, 2.0. Text columns like Gender (type=object) and integer columns like FCVC are excluded automatically.
<code>.columns</code>	After select_dtypes gives us a filtered DataFrame, .columns gives us just the column names as an Index object. Example result: Index(['Age', 'Height', 'Weight', 'NCP', 'CH2O', 'FAF', 'TUE'])
<code>.tolist()</code>	Converts the Index object into a plain Python list. Result: ['Age', 'Height', 'Weight', 'NCP', 'CH2O', 'FAF', 'TUE'] We need a plain list because we will use it later to select columns with data[continuous_columns].
<code>scaler = StandardScaler()</code>	Creates a StandardScaler object and saves it in a variable called 'scaler'. Right now it is empty — like buying a calculator but not pressing any buttons yet.

Code Piece	Plain-English Explanation + Example
<code>scaler.fit_transform(data[continuous_columns])</code>	<code>data[continuous_columns]</code> → selects only the float columns from the table. <code>fit</code> → the scaler reads those columns and calculates: the MEAN (average) and STD (standard deviation) of each column. <code>transform</code> → applies the formula: $\text{new_value} = (\text{original_value} - \text{mean}) / \text{std}$ Example: Height mean=1.70, std=0.09 → person with height 1.62 becomes $(1.62 - 1.70) / 0.09 = -0.88$ Result: every column now has mean≈0 and std≈1. A NumPy array (no column names yet).
<code>pd.DataFrame(scaled_features, columns=...)</code>	The <code>fit_transform</code> result is a plain NumPy array — just raw numbers, no column names. <code>pd.DataFrame(...)</code> wraps it back into a proper Pandas table. <code>columns=...</code> restores the original column names so we know which column is which.
<code>scaler.get_feature_names_out(continuous_columns)</code>	Returns the column names to use for the new scaled DataFrame. Example output: <code>array(['Age', 'Height', 'Weight', 'NCP', 'CH2O', 'FAF', 'TUE'])</code>
<code>pd.concat([...drop..., scaled_df], axis=1)</code>	<code>pd.concat</code> glues two DataFrames together. <code>axis=1</code> means 'join them side by side (column-wise)'. <code>data.drop(columns=continuous_columns)</code> → the original table with the OLD un-scaled float columns removed. <code>scaled_df</code> → the NEW scaled version of those columns. Result: same table as before, but the number columns are now normalised.

Before and after scaling example:

Height: 1.62 → -0.88 | Weight: 64 → -0.75 | Age: 21 → -0.52

The actual number doesn't matter anymore — what matters is how many standard deviations away from the average each person is.

Step 2 — One-Hot Encoding (OneHotEncoder)

Why? Columns like Gender contain text ('Male'/'Female'). Models only understand numbers. One-Hot Encoding converts each category into a separate 0/1 column.

```

categorical_columns =
scaled_data.select_dtypes(include=['object']).columns.tolist()
categorical_columns.remove('NObeyesdad') # keep the target out

encoder = OneHotEncoder(sparse_output=False, drop='first')
encoded_features = encoder.fit_transform(scaled_data[categorical_columns])

encoded_df = pd.DataFrame(encoded_features,
                           columns=encoder.get_feature_names_out(categorical_columns))

prepped_data = pd.concat([scaled_data.drop(columns=categorical_columns),
                           encoded_df], axis=1)

```

Code Piece	Plain-English Explanation + Example
<code>select_dtypes(include=['object'])</code>	Selects only columns whose type is 'object' (i.e., text/string columns). In Pandas, any column holding text is stored as dtype 'object'. Example result: ['Gender','CAEC','SMOKE','SCC','CALC','MTRANS','NO beyesdad']
<code>categorical_columns.remove('NObeyesdad')</code>	We do NOT want to encode the target column (what we are predicting). We handle it separately later. .remove(value) deletes one specific item from a Python list by its value — not by its position. After this line: ['Gender','CAEC','SMOKE','SCC','CALC','MTRANS']
<code>OneHotEncoder(sparse_output=False, drop='first')</code>	Creates the encoder tool with two settings: sparse_output=False → return a regular NumPy array (not a compressed sparse matrix, which is harder to work with). drop='first' → for each category column, drop the FIRST category to avoid redundancy. Example: Gender has Male/Female. With drop='first' we only need one column: Gender_Male (1=Male, 0=Female).
<code>encoder.fit_transform(scaled_data[categorical_columns])</code>	fit → the encoder learns all unique categories in each column. transform → converts each category to a row of 0s and 1s. Example: CAEC column has 4 values: Always/Frequently/Sometimes/no. After encoding with drop='first': 3 new columns appear: CAEC_Frequently, CAEC_Sometimes, CAEC_no. A person who snacks 'Sometimes' gets:

Code Piece	Plain-English Explanation + Example
	CAEC_Frequently=0, CAEC_Sometimes=1, CAEC_no=0
<code>encoder.get_feature_names_out(categorical_columns)</code>	Returns the new column names like: 'Gender_Male', 'CAEC_Frequently', 'CAEC_Sometimes', etc. These become the column labels of the encoded DataFrame.
<code>pd.concat([scaled_data.drop(columns=categorical_columns), encoded_df], axis=1)</code>	Same pattern as before: drop the original text columns and glue the new 0/1 columns in their place. The result is <code>prepped_data</code> — a fully numerical DataFrame ready for the model. Total columns expand from 17 to 24 because multi-category columns create multiple new columns.

Step 3 — Encoding the Target Variable

```
prepped_data['NObeyesdad'] =
prepped_data['NObeyesdad'].astype('category').cat.codes
prepped_data.head()
```

Code Piece	Plain-English Explanation
<code>prepped_data['NObeyesdad']</code>	Selects just the 'NObeyesdad' column from the DataFrame. Square brackets [] on a DataFrame with a column name returns that single column (called a Series).
<code>.astype('category')</code>	Converts the column's data type from 'object' (raw text) to Pandas 'category' type. This tells Pandas: 'these text values are categories, not just random strings.' Think of it like choosing 'category' type in a data entry form.
<code>.cat.codes</code>	After converting to 'category' type, <code>.cat</code> gives access to category-specific tools. <code>.codes</code> automatically assigns an integer to each unique category, sorted alphabetically. Example: Insufficient_Weight=0, Normal_Weight=3, Obesity_Type_I=4, Obesity_Type_II=5, Obesity_Type_III=6, Overweight_Level_I=1, Overweight_Level_II=2 Now the target column contains 0–6 instead of text

Code Piece	Plain-English Explanation
	strings.

Separating Features (X) and Target (y)

```
X = prepped_data.drop('NObeyesdad', axis=1)    # all columns EXCEPT the
target
y = prepped_data['NObeyesdad']                # ONLY the target column
```

Code Piece	Plain-English Explanation
<code>X = prepped_data.drop('NObeyesdad', axis=1)</code>	Creates DataFrame X with every column EXCEPT 'NObeyesdad'. axis=1 means 'drop something column-wise'. If you wrote axis=0, it would drop a ROW. X is the model's INPUT — all the information about each person.
<code>y = prepped_data['NObeyesdad']</code>	Creates Series y containing only the target column. y is what the model is trying to PREDICT — the correct obesity level for each person. Analogy: X is a student's answer sheet. y is the teacher's answer key.

Section 6 — Train / Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Parameter	What it means and why it matters
<code>X, y</code>	The input features and correct labels to split.
<code>test_size=0.2</code>	20% of the data goes into the test set. 80% goes into training. With 2111 rows: 1689 rows for training, 422 rows for testing.
<code>random_state=42</code>	A seed number that makes the random split reproducible every time. If two people both use <code>random_state=42</code> , they will get the exact same split. The number 42 has no special meaning — it is just a common convention.
<code>stratify=y</code>	Makes sure that the proportion of each obesity class is the same in BOTH train and test sets. Example: If 15% of all rows are 'Normal_Weight', then exactly 15% of both the training set and test set will be 'Normal_Weight'. Without this, bad luck could mean one class is almost missing from the test set.
<code>X_train, X_test, y_train, y_test =</code>	Python lets you unpack multiple return values in one line. <code>train_test_split</code> returns 4 items in this order: training features, test features, training labels, test labels. We name all four in one line.

Why split at all?

We train the model on training data and test it on data it has NEVER seen. This simulates the real world — where you train a model today and use it on new patients tomorrow. If you tested on training data, the model would score 99% just by memorising answers, not actually learning.

Section 7 — Logistic Regression: One-vs-All (OvA)

7.1 — How OvA Works

With 7 obesity classes, OvA trains **7 separate binary classifiers**:

- Classifier 1: Is this person 'Insufficient_Weight'? Yes vs No
- Classifier 2: Is this person 'Normal_Weight'? Yes vs No
- ... and so on for all 7 classes

At prediction time, all 7 classifiers score the new person. The class whose classifier gives the **highest confidence score** wins.

7.2 — Training the OvA Model

```
model_ova = LogisticRegression(multi_class='ovr', max_iter=1000)
model_ova.fit(X_train, y_train)
```

Code Piece	Plain-English Explanation
<code>LogisticRegression(multi_class='ovr', max_iter=1000)</code>	Creates a Logistic Regression model object with these settings: multi_class='ovr' → ovr stands for One-vs-Rest (same as One-vs-All). This instructs the model to use the OvA strategy. max_iter=1000 → Maximum iterations the optimisation algorithm can run. 1000 gives it enough time to find good weights. Default is 100, which sometimes isn't enough.
<code>model_ova.fit(X_train, y_train)</code>	.fit() is the TRAINING command. You give it the training features (X_train) and correct labels (y_train). The model then adjusts its internal weights repeatedly until it can correctly classify as many training examples as possible. This is the most important line in any ML project — this is where the model actually learns.

7.3 — Making Predictions and Measuring Accuracy

```
y_pred_ova = model_ova.predict(X_test)

print('One-vs-All (OvA) Strategy')
print(f'Accuracy: {np.round(100*accuracy_score(y_test, y_pred_ova),2)}%')
```

Code Piece	Plain-English Explanation
<code>model_ova.predict(X_test)</code>	Feeds the 422 test rows through the trained model. Returns an array of predicted class labels — one prediction per row. Example output: [3, 4, 1, 3, 6, 0, 2, ...] (numbers 0-6 representing the 7 obesity levels)
<code>accuracy_score(y_test, y_pred_ova)</code>	Compares the true labels (y_test) with the predicted labels (y_pred_ova). Counts how many predictions are correct and divides by total predictions. Returns a decimal: 0.87 = 87% of the 422 test rows were predicted correctly.
<code>100 * accuracy_score(...)</code>	Multiplies by 100 to convert decimal (0.87) to percentage (87.0).
<code>np.round(..., 2)</code>	<code>np.round(value, 2)</code> rounds to 2 decimal places. Example: <code>np.round(87.1234, 2) = 87.12</code>
<code>f"Accuracy: {np.round(...)}%"</code>	An f-string (formatted string). The f before the opening quote tells Python: insert variable values inside {}. Example: <code>score=87.25</code> → <code>f"Accuracy: {score}%"</code> outputs: Accuracy: 87.25%

Section 8 — Logistic Regression: One-vs-One (OvO)

8.1 — How OvO Works

OvO trains a classifier for **every possible PAIR** of classes:

- Classifier A: Normal_Weight vs Insufficient_Weight
- Classifier B: Normal_Weight vs Obesity_Type_I
- Classifier C: Obesity_Type_I vs Obesity_Type_II
- ... all 21 pairs ($7 \times 6 \div 2 = 21$ classifiers)

Each classifier votes for one of its two classes. The class that wins the **most votes** across all 21 classifiers is the final prediction.

8.2 — Training the OvO Model

```
model_ovo = OneVsOneClassifier(LogisticRegression(max_iter=1000))
model_ovo.fit(X_train, y_train)
```

Code Piece	Plain-English Explanation
<code>OneVsOneClassifier(LogisticRegression(max_iter=1000))</code>	OneVsOneClassifier is a WRAPPER. You pass any binary classifier inside it. Here we pass LogisticRegression(max_iter=1000) as the base classifier. The wrapper will automatically create 21 copies of LogisticRegression — one for each pair of classes. Think of it as a factory that mass-produces classifiers.
<code>model_ovo.fit(X_train, y_train)</code>	Trains all 21 binary classifiers at once. For each pair (e.g., Obesity_Type_I vs Normal_Weight), it trains only on rows belonging to those two classes, ignoring all other rows. This is why OvO can be more accurate — each classifier deals with a simpler, cleaner problem.

8.3 — OvA vs OvO Comparison

Feature	OvA vs OvO
Number of classifiers	OvA: k classifiers (7 for this dataset) OvO: $k(k-1)/2$ classifiers (21 for this dataset)

Feature	OvA vs OvO
Training speed	OvA is faster — fewer models to train.
Accuracy	OvO can be more accurate because each classifier focuses on just 2 classes.
Class imbalance	OvA is more affected — one class vs ALL others creates unbalanced binary problems.
Memory	OvO uses more memory — 21 models stored vs 7.

Section 9 — Exercises & Solutions

Exercise Q1 — Different Test Sizes

```
for test_size in [0.1, 0.3]:
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=42, stratify=y
    )
    model_ova.fit(X_train, y_train)
    y_pred = model_ova.predict(X_test)
    print(f'Test Size: {test_size}')
    print('Accuracy:', accuracy_score(y_test, y_pred))
```

Code Piece	Plain-English Explanation
<code>for test_size in [0.1, 0.3]:</code>	A Python for loop. The list [0.1, 0.3] has 2 items. The loop runs the indented block TWICE. First run: test_size = 0.1 (10% test data). Second run: test_size = 0.3 (30% test data). The colon at the end is mandatory — it signals the start of the loop body.
<code>(indented lines)</code>	In Python, indentation (4 spaces) defines what is INSIDE the loop. Every line indented under the for statement runs as part of the loop. When indentation stops, the loop ends.

Exercise Q2 — Feature Importance from Ova

```
feature_importance = np.mean(np.abs(model_ova.coef_), axis=0)
plt.barh(X.columns, feature_importance)
plt.title('Feature Importance')
plt.xlabel('Importance')
plt.show()
```

Code Piece	Plain-English Explanation
<code>model_ova.coef_</code>	The model's learned WEIGHTS (coefficients) — one row per class, one column per feature. Shape: (7, 24) meaning 7 rows (one per obesity level) and 24 columns (one per feature). Positive weight = that feature pushes toward this class. Negative = pushes away.
<code>np.abs(model_ova.coef_)</code>	Takes the ABSOLUTE VALUE of every weight —

Code Piece	Plain-English Explanation
	removes the minus sign. We care about the SIZE of the effect, not its direction. Example: -3.5 and +3.5 both become 3.5
<code>np.mean(..., axis=0)</code>	Computes the average DOWN the rows (axis=0 = 'across all 7 class rows'). Result: one single importance score per feature (24 values total). axis=0 → average down rows (result has same number of columns). axis=1 → average across columns (result has same number of rows).
<code>plt.barh(X.columns, feature_importance)</code>	Draws a HORIZONTAL bar chart. X.columns → the feature names (labels on y-axis). feature_importance → the values (lengths of the bars on x-axis).
<code>plt.xlabel('Importance')</code>	Adds a label to the x-axis.

Exercise Q2 — Feature Importance from OvO

```

coefs = np.array([est.coef_[0] for est in model_ovo.estimators_])
feature_importance = np.mean(np.abs(coefs), axis=0)
plt.barh(X.columns, feature_importance)
plt.title('Feature Importance (One-vs-One)')
plt.xlabel('Importance')
plt.show()

```

Code Piece	Plain-English Explanation
<code>model_ovo.estimators_</code>	A Python list containing all 21 trained binary classifiers inside the OvO wrapper. Each item in this list is a separate LogisticRegression object.
<code>[est.coef_[0] for est in model_ovo.estimators_]</code>	This is a LIST COMPREHENSION — a compact way to build a list. It means: 'For every classifier (est) in the list of estimators, take est.coef_[0]'. est.coef_ = the weight matrix for that binary classifier. [0] = take the first (and only) row. Result: a Python list of 21 arrays, each of length 24 (one weight per feature).
<code>np.array([...])</code>	Converts the Python list of 21 arrays into a single 2D

Code Piece	Plain-English Explanation
	NumPy array of shape (21, 24). 21 rows = 21 classifiers. 24 columns = 24 features. This is needed so we can apply np.mean across all rows.
<code>np.mean(np.abs(coefs), axis=0)</code>	Same as OvA: take absolute values of all weights, then average across all 21 classifiers. Result: 24 importance scores — one per feature.

List Comprehension Syntax:

`[expression for variable in iterable]`

Example: `[x*2 for x in [1,2,3]]` produces `[2, 4, 6]`

It is exactly the same as this longer version:

```
result = []
for x in [1, 2, 3]:
    result.append(x * 2)
# result = [2, 4, 6]
```

Section 10 — Exercise Q3: Full Pipeline Function

A function bundles all steps into one reusable block you can call with one line.

```
def obesity_risk_pipeline(data_path, test_size=0.2):
    # 1. Load data
    data = pd.read_csv(data_path)

    # 2. Scale numerical features
    continuous_columns =
data.select_dtypes(include=['float64']).columns.tolist()
    scaler = StandardScaler()
    scaled_features = scaler.fit_transform(data[continuous_columns])
    scaled_df = pd.DataFrame(scaled_features,

columns=scaler.get_feature_names_out(continuous_columns))
    scaled_data = pd.concat([data.drop(columns=continuous_columns),
scaled_df], axis=1)

    # 3. One-hot encode categorical features
    categorical_columns =
scaled_data.select_dtypes(include=['object']).columns.tolist()
    categorical_columns.remove('NObeyesdad')
    encoder = OneHotEncoder(sparse_output=False, drop='first')
    encoded_features =
encoder.fit_transform(scaled_data[categorical_columns])
    encoded_df = pd.DataFrame(encoded_features,

columns=encoder.get_feature_names_out(categorical_columns))
    prepped_data =
pd.concat([scaled_data.drop(columns=categorical_columns), encoded_df],
axis=1)

    # 4. Encode target
    prepped_data['NObeyesdad'] =
prepped_data['NObeyesdad'].astype('category').cat.codes

    # 5. Split
    X = prepped_data.drop('NObeyesdad', axis=1)
    y = prepped_data['NObeyesdad']
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=42, stratify=y)

    # 6. Train and evaluate
    model = LogisticRegression(multi_class='multinomial', max_iter=1000)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print('Accuracy:', accuracy_score(y_test, y_pred))

# Call the function
obesity_risk_pipeline(file_path, test_size=0.2)
```

Code Piece	Plain-English Explanation
<pre>def obesity_risk_pipeline(data_ path, test_size=0.2):</pre>	def → keyword that DEFINES a function. obesity_risk_pipeline → the function name (you can call it anything). data_path → a required parameter. You must provide this when calling the function. test_size=0.2 → an OPTIONAL parameter with a default value. If you do not provide it, Python uses 0.2 automatically. The colon : at the end is mandatory.
<pre>(4-space indented body)</pre>	Everything inside the function must be indented by 4 spaces. When indentation returns to 0, the function definition ends.
<pre>multi_class='multinomial'</pre>	A different strategy from 'ovr'. Multinomial considers all classes at the same time (not one-vs-rest). It trains a single combined model that assigns probabilities across all 7 classes simultaneously. Often gives slightly better accuracy when classes are balanced.
<pre>obesity_risk_pipeline(file_ path, test_size=0.2)</pre>	CALLS (runs) the function. file_path → the value passed into data_path parameter. test_size=0.2 → the value passed into the test_size parameter. All 6 steps inside the function execute automatically.

Section 11 — Quick-Reference: Key Syntax

Python Syntax	What it does — one-line explanation
<code>df['column_name']</code>	Selects a single column from a DataFrame. Returns a Series (a single list of values).
<code>df[['col1', 'col2']]</code>	Selects MULTIPLE columns. Outer <code>[]</code> selects from the DataFrame. Inner <code>[]</code> is a Python list of names. Result is a DataFrame.
<code>df.drop(columns=['col'], axis=1)</code>	Removes the specified column(s). <code>axis=1</code> = remove columns. <code>axis=0</code> = remove rows.
<code>df.iloc[row_num, col_num]</code>	Integer Location. Selects by POSITION (0-based index). <code>df.iloc[0, 2]</code> = first row, third column. Use when you know the position but not the name.
<code>df.loc[row_label, col_name]</code>	Label Location. Selects by LABEL (actual name). <code>df.loc[0, 'Height']</code> = row labelled 0, column named 'Height'. Use when you know the column name. Preferred for clarity.
<code>axes.flatten()</code>	When <code>plt.subplots(2,3)</code> creates a 2D grid of subplot axes, <code>flatten()</code> converts it to a 1D list. This lets you loop through all subplots with a single simple for loop instead of nested loops. Analogy: turns a 2×3 egg carton into a single row of 6 eggs.
<code>pd.concat([df1, df2], axis=1)</code>	Glues DataFrames together. <code>axis=1</code> = side by side (column-wise). <code>axis=0</code> = stacked (row-wise).
<code>np.abs(array)</code>	Returns absolute value (no minus sign) for every element. <code>-3.5 → 3.5</code>
<code>np.mean(array, axis=0)</code>	Computes average. <code>axis=0</code> averages DOWN rows. <code>axis=1</code> averages ACROSS columns.
<code>f"text {variable} more text"</code>	f-string: puts a variable's value directly inside a string. The <code>f</code> before the quote activates it.
<code>[expr for x in iterable]</code>	List comprehension: compact loop that builds a list. Equivalent to a for loop with <code>.append()</code> .

Section 12 — Full Pipeline Summary

Step	What happens
1. Import libraries	Load pandas, numpy, matplotlib, seaborn, and scikit-learn tools.
2. Load data	<code>pd.read_csv()</code> reads the CSV into a 2111×17 DataFrame.
3. EDA	<code>countplot</code> shows class distribution. <code>isnull().sum()</code> confirms no missing values.
4. Feature Scaling	<code>StandardScaler</code> transforms all float columns to mean=0, std=1 so they are comparable.
5. One-Hot Encoding	<code>OneHotEncoder</code> converts all text columns to binary (0/1) columns. Total columns: 17 → 24.
6. Encode Target	<code>.astype('category').cat.codes</code> converts the 7 obesity level names to integers 0–6.
7. Train/Test Split	<code>train_test_split</code> holds out 20% of data (422 rows) for testing. <code>stratify=y</code> keeps class balance.
8. Train OvA Model	<code>LogisticRegression(multi_class='ovr')</code> trains 7 binary classifiers (one per class).
9. Train OvO Model	<code>OneVsOneClassifier(LogisticRegression())</code> trains 21 pairwise classifiers.
10. Evaluate	<code>accuracy_score(y_test, y_pred)</code> gives the percentage of correct predictions on unseen data.

Key Takeaway:

Multi-class classification breaks a hard problem (7 classes) into many simple Yes/No problems.

One-vs-All: asks 'Is it THIS class?' 7 times. One-vs-One: asks 'Is it A or B?' for all 21 pairs.

Both approaches transform the same complex dataset into binary problems that Logistic Regression handles beautifully.